

SATELLITE IMAGE THREAT DETECTION (SITP)

LOW-LEVEL DETAILED DESIGN SPECIFICATION (LLD)

Document ID:	SITP-LLD-v3.2.0	Classification:	GEOINT Technical Engineering Spec
Lead Module:	src.SITP & application.py	Cloud Infrastructure:	AWS IAM, ECR, ECS Fargate & EC2
Topics:	Class Interfaces, Algorithms, IAM JSON, API Contracts	Release Date:	September 2026

1. Modular Component Implementations

A. DataIngestion Component (src/SITP/components/data_ingestion.py)

Class: DataIngestion(config: DataIngestionConfig)
Method: initiate_data_ingestion() -> tuple[pd.DataFrame, dict, Path, Path]
Internal Logic: Parses GeoJSON feature collections, extracts exterior polygon bounds [min_x, min_y, max_x, max_y], maps raw xView category IDs to 62 contiguous indices, and executes an 80/20 train/validation split at the image level using a fixed pseudo-random seed (42) to eliminate spatial data leakage.

B. DataTransformation Component (src/SITP/components/data_transformation.py)

Class: DataTransformation(config: DataTransformationConfig)
Method: initiate_data_transformation(ann_df, split, img_dir, out_dir) -> tuple[Path, dict]
Tiling Algorithm: Generates 512x512 pixel chips with stride $S = T - O = 512 - 100 = 412$ px. Normalizes bounding boxes to YOLO format: [class_idx, x_center, y_center, w, h] in [0, 1]. Filters truncated boxes below a min area threshold.

C. ModelTrainer Component (src/SITP/components/model_trainer.py)

Class: ModelTrainer(config: ModelTrainerConfig)
Method: initiate_model_trainer(data_yaml: Path, output_dir: Path) -> tuple[YOLO, Path]
Hyperparameters: Epochs=50, Imgsz=512, Batch=16, Optimizer=AdamW, Cosine LR (lr0=0.01, lrf=0.01), Mosaic=1.0, STAL assignment.

2. PredictPipeline & ModelMonitoring Detailed Design

A. PredictPipeline Model Engine Auto-Select (src/SITP/pipelines/prediction_pipeline.py)

Class: PredictPipeline(model_path=None, conf=0.25, iou=0.45, batch_size=2)

Auto-Select Hierarchy: 1) Checks for 'best-v26.pt' or 'best.onnx' in project root. 2) If .onnx is detected, initializes YOLO('best.onnx', task='detect') utilizing ONNX Runtime (CPUExecutionProvider). 3) Otherwise falls back to PyTorch (best-v26.pt / best.pt). The active engine is exported as self.engine_type.

B. Subsampled Statistical Variance Gating Algorithm

To prevent executing deep learning forward passes on empty ocean, void borders, or solid cloud, SITP evaluates subsampled tile statistics:

```
# Algorithmic Step: Subsample tile at 4-pixel spatial stride
sample = chip[:, :4, :4]
sample_std = float(np.std(sample))
sample_mean = float(np.mean(sample))
# Rejection Condition 1: Featureless ocean / uniform ground
if sample_std < 5.0: skip_tile()
# Rejection Condition 2: Sensor void / border padding
elif sample_mean < 4.0 and sample_std < 3.0: skip_tile()
# Rejection Condition 3: Saturated cloud whiteout
elif sample_mean > 245.0 and sample_std < 4.0: skip_tile()
```

C. Global Coordinate Projection & Deduplication

Local tile coordinates [x1, y1, x2, y2] from batched forward passes are projected to global GeoTIFF pixel space via: $X_{global} = x_{local} + x_{offset}$, $Y_{global} = y_{local} + y_{offset}$. Overlapping tile boundary detections are merged using IoU thresholding ($\tau_{iou} = 0.45$).

3. Mathematical Formulations & Loss Architecture

Small-Target-Aware Label Assignment (STAL): Enhances standard IoU cost formulations by weighting center-distance and normalized scale disparity: $C_{STAL} = \alpha * C_{cls} + \beta * C_{box} + \gamma * (1 - \exp(-d^2 / (2 * \sigma_s^2)))$, where σ_s dynamically scales with target bounding box diagonal, granting sub-32px objects higher candidate retention.

4. API Schemas & Service Endpoints

SITP exposes lightweight, production-hardened REST endpoints via Flask:

Endpoint	Method	Request Payload	Response Specification
/	GET	None	HTML5 Tactical HUD landing interface with radar telemetry and threshold sliders.
/	POST	multipart/form-data: - file: GeoTIFF (.tif) - conf: float [0..1] - iou: float [0..1]	Rendered HUD with 8 analytical result panels: stats bar, engine badge, threat brief, latency breakdown, image metadata, overlay, class chart, detection table.
/health	GET	None	HTTP 200 JSON payload: {status: 'ok', service: 'satellite-threat-detection', deployment: 'active'}. Responds in <10 ms.

5. High-Threat Intelligence Brief Generation Algorithm

The web application automatically derives natural-language tactical dossiers from raw inference arrays: detections with confidence ≥ 0.75 (HIGH_CONF_THRESHOLD) are aggregated by classified category name and sorted by frequency. Three intelligence conditions are dynamically evaluated: 1) Zero High-Confidence Targets: Emits advisory to lower confidence slider for reconnaissance scan. 2) Single Dominant Threat: Highlights tactical count and localized class. 3) Multi-Class Threat Array: Emits formatted alert tagging all strategic objects.

6. AWS IAM Security Policies & Role Specifications

To adhere to enterprise defense and zero-trust standards, SITP configures explicit IAM role separation:

IAM Entity	Principal / Service	Required Actions / Managed Policy
sitp-ecs-task-execution-role	ecs-tasks.amazonaws.com	AmazonECSTaskExecutionRolePolicy (ecr:GetAuthorizationToken, ecr:BatchCheckLayerAvailability, ecr:GetDownloadUrlForLayer, ecr:BatchGetImage, logs:CreateLogStream, logs:PutLogEvents)
sitp-ecs-task-role	ecs-tasks.amazonaws.com	Scoped Least Privilege: s3:GetObject on input raster bucket, s3:PutObject on telemetry archive bucket, cloudwatch:PutMetricData for inference latency metrics.
sitp-ec2-instance-role	ec2.amazonaws.com	AmazonEC2ContainerServiceforEC2Role (registers EC2 node with ECS cluster) and AmazonSSMManagedInstanceCore (secure Systems Manager shell without SSH port 22).

7. Amazon ECS Task Definition Specification (AWS Fargate / EC2)

Task definitions are parameterized for either serverless AWS Fargate or dedicated Amazon EC2 GPU instances:

```
{
  "family": "sitp-task-definition",
  "networkMode": "awsvpc",
  "executionRoleArn": "arn:aws:iam::role/sitp-ecs-task-execution-role",
  "taskRoleArn": "arn:aws:iam::role/sitp-ecs-task-role",
  "requiresCompatibilities": ["FARGATE", "EC2"],
  "cpu": "1024",
  "memory": "2048",
  "containerDefinitions": [{
    "name": "sitp-container",
    "image": ".dkr.ecr.amazonaws.com/satellite-image-threat-detection:latest",
    "portMappings": [{"containerPort": 10000, "protocol": "tcp"}],
    "environment": [
      {"name": "FLASK_APP", "value": "application.py"},
      {"name": "PORT", "value": "10000"},
      {"name": "WEB_CONCURRENCY", "value": "1"}
    ],
    "healthCheck": {
      "command": ["CMD-SHELL", "curl -f http://localhost:10000/health || exit 1"],
      "interval": 30, "timeout": 5, "retries": 3, "startPeriod": 60
    }
  ]
}
```